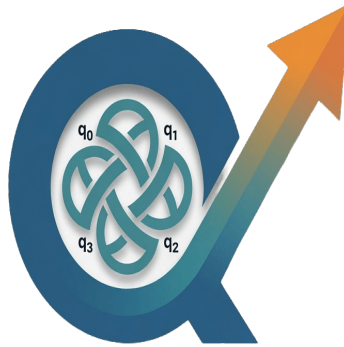




The Mathematics of QATQ

Exact Tensor-Aware Compression with Quaternion-Structured Research Predictors

Codec specification aligned with the current Rust implementation

Production surface: [qatq-exact](#) and the sequential [QATC](#) container

QATQ is best modelled as an **adaptive exact encoder**, not as a single lossy quaternion-vector quantizer. It evaluates a family of reversible representations and selects a smallest valid payload. Quaternion rotation and QJL residual signs remain useful experimental comparators, but neither is the source of the production codec's exactness.

Version 1.0 | 25 July 2026

Grounded in the QATQ repository implementation and technical whitepaper.

1 Scope and Exactness Contract

Let a tensor contain n native elements of dtype t in $\{f32, f16, bf16\}$. Let $B_t(x)$ be the exact little-endian byte string supplied by the caller. QATQ's production contract is:

$$D_t(E_t(B_t(x))) = B_t(x)$$

Equality is byte-for-byte. Consequently, for f32 it preserves every 32-bit pattern, including positive and negative zero, infinities, and NaN payload bits. For f16 and bf16 it preserves the native 16-bit elements without widening them.

Canonicalization. Internally, bytes may be reordered into a canonical big-endian element representation:

$$C_t : B_t(x) \rightarrow c, C_t^{-1}(c) = B_t(x)$$

Canonicalization is a permutation of bytes, not a floating-point conversion. It is therefore invertible for every possible native bit pattern.

Adaptive selection. Let $S(t, x)$ be the strategies applicable to the dtype and API path. For each strategy s :

$$E_s : c \rightarrow p_s, D_s(p_s) = c$$

$$s^* = \arg \min_{\{s \in S(t, x)\}} |header + p_s|$$

The encoder writes the selected strategy identifier and its payload into a QATQ envelope. A raw-bit candidate is always available, so correctness does not depend on the input being compressible.

Production storage decision.

If the best exact representation is compression-positive, the decision is **Compressed**. If raw storage is preferable, the production API may return **PassThroughRaw**. This distinguishes exact encodability from a useful compression decision.

2 Exact Strategy Family

The f32 exact search can compare the following representations. Typed f16/bf16 paths use the compatible byte-oriented subset. All transforms in this table are reversible before checksum validation.

Strategy	Reversible transform	Final coding
Raw bits	Identity	None
Byte RLE	Identity	Zero/raw runs
Byte-plane family	Transpose bytes by significance	RLE, packed RLE, blocks, or zstd
Delta-XOR	Adjacent word XOR	Byte-plane RLE
Quaternion-chain	Wrapping word deltas in lanes of four	Byte-plane zstd
Phase 1 predictor + XOR	Lossy quaternion predictor plus exact XOR residual	Run-coded residual

Byte-plane permutation.

For n words of width w bytes, let $c[i,j]$ be byte j of element i in canonical order. The byte-plane transform concatenates equal-significance bytes:

$$P(c) = (c[0,0], \dots, c[n-1,0], c[0,1], \dots, c[n-1,w-1])$$

P is a permutation, so $P^{-1}(P(c)) = c$. Separate planes often expose zero, repeated, or low-entropy regions in floating-point tensors.

Adjacent delta-XOR.

For f32 bit words $u_0, \dots, u_{n-1}$, define $d_0 = u_0$ and $d_i = u_i \text{ XOR } u_{i-1}$. Decode by:

$$u_0 = d_0; \quad u_i = d_i \text{ XOR } u_{i-1}, \quad i > 0$$

The residual words are byte-plane ordered and run coded. XOR operates on bit patterns, so it remains exact for all IEEE-754 encodings.

3 Reversible Quaternion-Chain Residual Transform

The production strategy called **quaternion-chain-zstd** groups f32 words into conceptual four-coordinate lanes. Its transform is an integer lifting scheme over bit patterns - not a quaternion spatial rotation and not a codebook quantizer.

Let $u_i = \text{bits}(x_i)$ be a 32-bit unsigned word and set $u_{-1} = 0$. The encoder stores wrapping differences in the ring $\mathbb{Z}/(2^{32}\mathbb{Z})$:

$$r_i = (u_i - u_{(i-1)}) \bmod 2^{32}$$

For a lane (a,b,c,d) following previous lane endpoint p, this is:

$$(a-p, b-a, c-b, d-c) \bmod 2^{32}$$

The chain carries d into the next lane's a. Decode uses wrapping addition:

$$u_i = (u_{(i-1)} + r_i) \bmod 2^{32}$$

By induction, decoding recovers every u_i exactly. The residual words are then transposed into byte planes and compressed with zstd. Entropy coding changes payload size but not the inverse relation.

Why retain the quaternion name? The grouping preserves the project's four-coordinate lane structure, while the actual exact transform is deliberately simpler and fully reversible. It should not be described as $q \times q^{-1}$, an $SO(4)$ rotation, or nearest-centroid quantization.

Selection condition.

This candidate is selected only if its complete strategy body is smaller than the competing exact candidates. Quaternion structure is therefore an empirical compression option, not a mandatory stage.

4 Quaternion Phase 1 Predictor with Exact XOR Closure

One exact strategy uses the lossy **phase1-q4** pipeline only to produce a predictor. An exact XOR residual then closes the gap. This cleanly separates prediction quality from correctness.

4.1 Lane rotation

Pad the final lane with zeros and map each four-vector to a quaternion $q_l = a + bi + cj + dk$. A deterministic unit quaternion h_l is derived from the stored seed and lane index. The forward and inverse maps are:

$$y_l = h_l q_l \text{ conjugate}(h_l)$$

$$q_l = \text{conjugate}(h_l) y_l h_l$$

For unit h_l , conjugation is norm preserving and fixes the real component while rotating the imaginary three-vector. It is not a general $SO(4)$ rotation.

4.2 Signed q4 predictor

$$\alpha = \max_j |y_j| / 7; k_j = \text{clip}(\text{round}(y_j/\alpha), -7, 7)$$

$$e_j = y_j - \alpha k_j; \rho = (1/m) \sum_j |e_j|$$

$$\hat{y}_j = \alpha k_j + \rho \text{ sign}(e_j)$$

The implementation stores packed q4 coordinates, one residual sign bit per rotated coordinate, the deterministic seed, and the global mean absolute residual magnitude ρ . Inverse quaternion conjugation produces predictor values $\hat{x}$.

4.3 Exact closure

$$z_i = \text{bits}(x_i) \text{ XOR } \text{bits}(\hat{x}_i)$$

$$\text{bits}(x_i) = z_i \text{ XOR } \text{bits}(\hat{x}_i)$$

The z_i stream is run coded. Even if the q4 predictor is inaccurate, the XOR identity guarantees exact recovery. The predictor affects compression ratio only; it cannot weaken the exactness contract.

5 Lossy Research Comparators

QATQ retains two explicitly lossy comparator modes. They are mathematically separate and must not be merged into the production codec specification.

5.1 phase1-q4

This mode uses the quaternion rotation and q4 residual-sign predictor from Section 4, but omits the exact XOR closure:

$$\tilde{x} = H^{(-1)}(\alpha k + \rho \text{sign}(e))$$

It is a reconstruction-error experiment. A sign bit plus one global magnitude cannot recover arbitrary residual coordinates exactly.

5.2 turboquant-q4 reference baseline

This local comparator pads the vector to a power-of-two dimension and applies a seeded randomized normalized Walsh-Hadamard transform R , signed q4 scalar quantization, and QJL-style signs of the residual projection:

$$y = R x; k = Q4(y); x_0 = R^{(-1)}(\alpha k)$$

$$e = x - x_0; b = \text{sign}(R_{\text{qjl}} e)$$

For query v , the implementation forms an approximate residual correction to $v^T x$ using projected query coordinates, the residual norm, and b . Schematically:

$$\hat{I}_{\text{hat}}(v, x) = v^T x_0 + [\sqrt{\pi/2}/m] \|e\|_2 \sum_{j=1}^m (R_{\text{qjl}} v)_j b_j$$

This is a randomized inner-product **estimator**. It does not guarantee exact inner products and the QJL sign vector does not reconstruct arbitrary residuals exactly. The mode is a local reference baseline, not an official TurboQuant implementation.

6 Container, Integrity, and Claim Boundaries

QATQ payload integrity.

A QATQ exact envelope records the mode, strategy metadata, element count, dtype where applicable, and a checksum of the original data. Decode is accepted only when structure, bounds, inverse transforms, and checksum validation succeed.

$$\text{Accept}(p) \text{ iff } \text{well_formed}(p) \text{ AND } \text{checksum}(D(p)) = \text{checksum}(\text{original})$$

Sequential QATC composition.

For a tensor partitioned into chunks $x^{(1)}, \dots, x^{(k)}$, QATC stores complete QATQ exact payloads $p_j = E(x^{(j)})$ with length framing and an aggregate checksum:

$$D_QATC(p_1, \dots, p_k) = D(p_1) \parallel \dots \parallel D(p_k)$$

QATC is sequential transport. It does not, by itself, define random-access cache semantics or prove live GPU residency reduction.

Accurate claims

- **Exact:** qatq-exact and QATC preserve supported native tensor bytes bit-for-bit.
- **Adaptive:** compression comes from selecting among reversible representations.
- **Quaternion-structured:** four-coordinate structure appears in an exact delta transform and in a lossy predictor.
- **Scoped:** benchmark results support the tested fixtures and runtime captures, not universal superiority.

Claims this specification does not make

- QJL signs preserve inner products exactly.
- Quaternion conjugation is an arbitrary four-dimensional rotation.
- The production codec uses a spherical centroid codebook or FP8/FP16 lane norms.
- Exact reconstruction follows from q4 quantization without an exact residual.
- The container alone proves live VRAM reduction or model-quality improvement.

Repository basis. QATQ README; docs/WHITEPAPER.md; docs/ARCHITECTURE.md; and the encode/decode implementation in src/lib.rs. The implementation is authoritative if this explanatory document and code diverge.